

A sales assistant that answers from the product documents, captures leads, and remembers every chat

Advanced RAG with LangChain and LangGraph · FAISS · MCP lead tool · per-session memory · React web chat. Runs 100% locally on Ollama.

100%

answer accuracy (18-question test)

14

node LangGraph with a retry cycle

32

automated tests

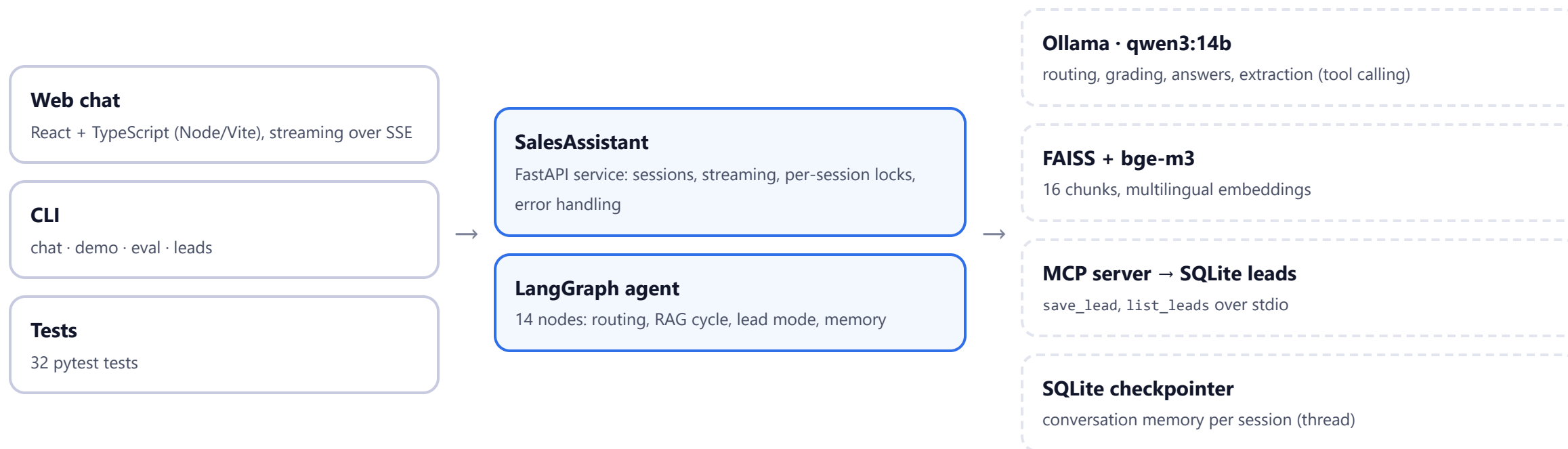
Every requirement, plus both bonus tasks

What the test case asked for, and how it is delivered

Requirement	Implementation
Framework	LangChain + LangGraph state machine: 14 nodes, conditional routing, a retrieve → grade → rewrite cycle
Vector database	FAISS (cosine similarity) over bge-m3 embeddings; Thai and English questions both work
Answers from 5 PDFs	16 chunks from 5 PDFs; every answer cites [file, page]
Not-found handling	An LLM relevance grader, one rewritten retry, then a polite fallback. It never invents an answer
Bonus 1 Lead collection	Interest triggers lead mode; name, occupation, income and phone go into a Pydantic model and are saved to SQLite through an MCP server
Bonus 2 Sessions	LangGraph SQLite checkpointer, one thread per session, plus a running summary for long chats. Chats can be listed, renamed and deleted
Deliverables	Source + requirements.txt · README (setup, tool usage, graph) · demo logs · evaluation report · web chat

One agent, three front doors, four local services

Everything runs on one machine; no data leaves it



Why local: customer data (names, incomes, phones) never goes to an external API, and there are no per-token costs.

From PDF to searchable vectors in one command

python main.py ingest builds the index; replacing the PDFs needs no code change

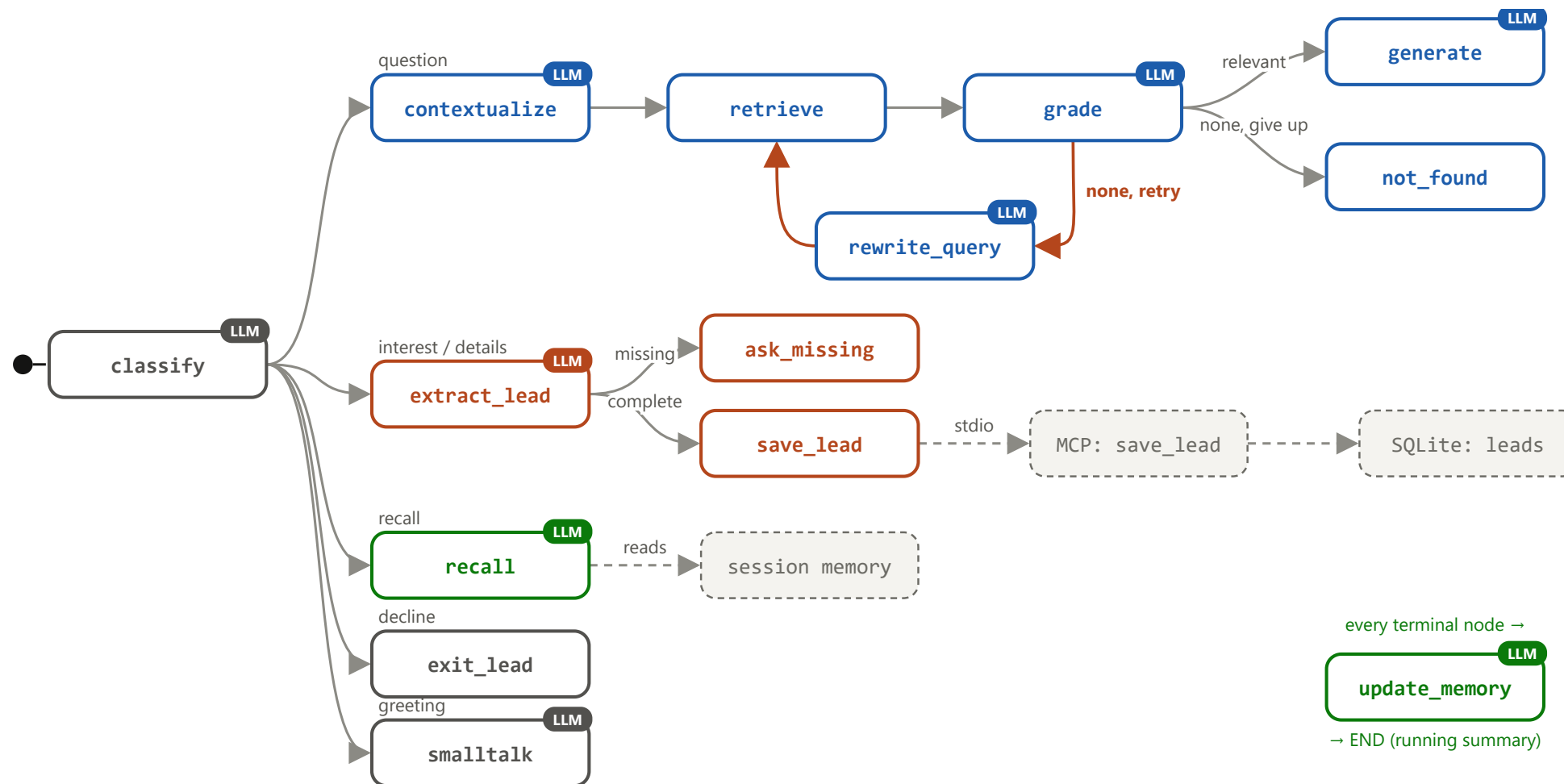


Document (sample)	Chunks
01_PA_Plus_Personal_Accident.pdf	3
02_Life_Secure_Term_Life.pdf	3
03_Health_Care_Plus.pdf	3
04_Sales_Guide_Underwriting_FAQ.pdf	4
05_Claims_and_Service_Guide.pdf	3

- 1 The real 5 PDFs were not provided, so I created **5 sample documents** with fictional products, prices and rules, clearly marked "SAMPLE".
- 2 **bge-m3** is multilingual: a **Thai question** finds the right passage in English documents (the evaluation includes 2 Thai questions).
- 3 Chunks keep their **file and page**, so every answer can cite its source.

The agent is a state machine, not a single prompt

The router picks a path each turn; LLM badges mark nodes that call the model; the orange loop is the retry cycle

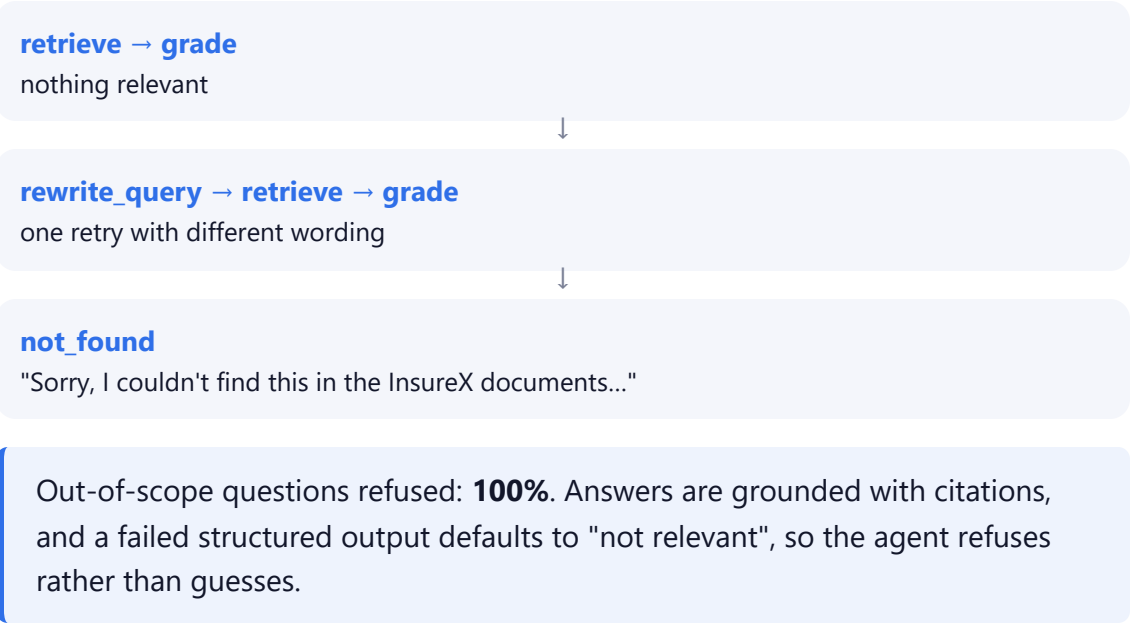


Similarity alone can't tell "not in the documents", so a grader decides

Top similarity scores for three questions; only one of them is unanswerable

"How much is PA Plus Gold per year?"		0.61
"Waiting period for cataract?"		0.58
"Do you sell car insurance?"		0.52

A threshold would either let car insurance through or block real questions. The **grade** node asks the LLM whether each passage *directly* answers the question.



Interest switches the agent into lead mode; MCP saves validated leads

Details can arrive over several messages, in English or Thai, mixed with questions

I'd like to apply for Health Care Plus

Great to hear you're interested in Health Care Plus! ...could you share your full name, occupation, monthly income and phone?

Dave Miller, developer, 85000 per month, phone 12345

Sorry, some details don't look right. Could you check the phone number?

Sorry, it's 091-555-0123

Thank you, Dave! Your details are saved (lead #3)...

Step	How
extract_lead	LLM tool call into a Pydantic LeadDraft; merged across turns
ask_missing	Asks only for what's still missing; side questions are answered first
save_lead	Calls the MCP tool save_lead (stdio)
Validation	Inside the tool: Thai phone format (normalised to 0XXXXXXXX), income > 0, names trimmed. If a field is rejected, only that field is asked again
Storage	SQLite leads table; list_leads reads it back

Every chat remembers its own conversation, even after a restart

LangGraph checkpoint: `thread_id = session_id`; state is saved after every node

Separation

Each session is its own thread. Bob asks "what did I ask before?" and gets "no record", while Alice's chat is untouched.

Recent + summary

The last 8 messages go to the LLM verbatim. Past 12 unsummarised messages, the `update_memory` node folds older ones into a **running summary**, so long chats aren't forgotten.

Persistence

SQLite on disk: close the app, reopen the chat, and it continues. Follow-ups like "and for medical expenses?" are resolved from memory.

- 1 Chats are listed on the server with an auto title and date; they can be **renamed** and **deleted** (delete erases the memory; saved leads stay as business records).
- 2 One turn at a time per session (async lock), so two browser tabs can't interleave one conversation.

Tested: after 14 messages the summary kept "Ploy, 32, pharmacist, PA Plus Silver 1,890 THB", and the agent answered "what was the first price you quoted me?" correctly.

A web chat that shows the agent working

React + TypeScript (Node.js / Vite) on a FastAPI backend; answers stream token by token

The screenshot displays a web chat application for 'InsureX Sales Assistant'. The interface is dark-themed and organized into three vertical panels.

Left Panel (Chat Management): Features a 'New chat' button, a search bar, and a list of chat sessions. The active session is 'Nok - PA Plus Gold (lead)' with 4 messages.

Center Panel (Conversation): Shows a dialogue between the user (Nok) and the assistant (IX). The user asks about the cost of PA Plus Gold and its medical coverage. The assistant provides detailed answers, including premium amounts and coverage limits, and offers to help with other products or provide contact information.

Right Panel (Agent Workflow & Lead Capture): Contains three main sections:

- Agent pipeline:** Shows the current step 'Collecting contact details'.
- Session memory:** Displays '8 messages remembered' and an 'Off' running summary.
- Lead capture:** A progress indicator shows '2/4' details collected. A table lists the captured information:

Field	Value	Status
Name	Nok Siriporn	✓
Occupation	teacher	✓
Income / month	-	
Phone	-	

What the scripted demo proves

17 turns across 4 customers and one restart; 3 leads saved (python main.py demo)

Session	Customer	Assistant	Graph path	Time
alice	How much does PA Plus Gold cost per year?	The annual premium for PA Plus Gold is 3,490 THB for occupation classes 1-2 and 4,590 THB for occupation class 3 [01_PA_Plus_Personal_Accident.pdf,...	classify → contextualize → retrieve → grade → generate → update_memory	4.7s
alice	And how much does it pay for medical expenses?	PA Plus Gold covers medical expenses up to 50,000 THB per accident [01_PA_Plus_Personal_Accident.pdf, p.1].	classify → contextualize → retrieve → grade → generate → update_memory	2.2s
alice	Do you sell car insurance?	Sorry, I couldn't find this in the InsureX product documents, so I can't give a reliable answer. I can help with PA Plus, Life Secure and Health Ca...	classify → contextualize → retrieve → grade → rewrite_query → retrieve → grade → not_found → update_memory	2.1s
alice	I earn about 32,000 baht a month, my number is 089-765-4321	Thank you, Alice Wong! Your details are saved (lead #1). An InsureX agent will call you on 0897654321 about PA Plus shortly.	classify → extract_lead → save_lead → update_memory	3.3s
somchai	Life Secure ลดหย่อนภาษีได้สูงสุดเท่าไร	Life Secure สามารถลดหย่อนภาษีได้สูงสุด 100,000 บาทต่อปี ตามกฎของกรมสรรพากร [02_Life_Secure_Term_Life.pdf, p.1]	classify → contextualize → retrieve → grade → generate → update_memory	4.5s
somchai	สนใจครับ ผมชื่อสมชาย ใจดี เป็นวิศวกร เงินเดือน 5 หมื่น เบอร์ 081-234-5678	ขอบคุณค่ะ คุณสมชาย ใจดี บันทึกข้อมูลเรียบร้อยแล้ว (หมายเลข 2) ตัวแทนจะติดต่อกลับที่เบอร์ 0812345678 เร็ว ๆ นี้ค่ะ	classify → extract_lead → save_lead → update_memory	2.7s
dave	Dave Miller, software developer, 85000 per month, phone 12345	Sorry, some details don't look right. Could you check a contact phone number again?	classify → extract_lead → save_lead → update_memory	2.4s
bob	What's my name? What did I ask you before?	I don't have any previous conversation history to reference.	classify → recall → update_memory	0.9s
alice	Can you remind me how much the Gold plan was?	The annual premium for PA Plus Gold is 3,490 THB for occupation classes 1-2 and 4,590 THB for occupation class 3.	classify → recall → update_memory	3.0s

RESULTS

Every test question answered correctly, every off-topic one refused

18 questions: 16 answerable (2 in Thai) and 2 out of scope (python main.py eval)

Retrieval hit rate

100%

right document in the top 4

Answer accuracy

100%

expected fact in the answer

Out of scope refused

100%

no invented answers

Avg. RAG answer

4.4s

RTX 4070 SUPER, local

Turn type (demo)	Turns	Avg	Range
Memory / small talk	4	1.7 s	0.9–3.0 s
RAG answer	5	4.4 s	2.2–5.4 s
Not found (with retry)	1	2.1 s	2.1–2.1 s
Lead question	3	1.4 s	1.4–1.5 s
Lead saved / validated	4	2.8 s	2.4–3.3 s

Caveat: the set is small and built on sample documents. With the real PDFs, I'd rebuild the question set with the business and track these numbers on every change.

Built like a product: small classes, safe defaults, tests

Each criterion in the brief, and how it is met

Structure (OOP)

- KnowledgeBase · FaissVectorStore · PdfLoader · TextChunker
- SalesAgent (graph) · SalesAssistant (service) · SessionIndex
- Lead / LeadDraft (Pydantic) · LeadRepository · MCP server/client
- All prompts in one module; settings overridable by environment variables

Robustness

- Structured output: tool calling → retry → JSON mode → safe default
- Any failure, including Ollama being down, returns a friendly message; the session survives
- Input validation on every API (session id pattern, length limits)
- Model output is rendered as text, never as HTML

Testing

- 32 pytest tests: routing, leads, FAISS, MCP over stdio, web API, sessions, end-to-end
- Evaluation set for RAG precision; scripted demo for the rest

Environment

- requirements.txt (pinned), .env.example, Python 3.12 venv
- One-click start_chatbot.bat; README covers setup, MCP tool and graph

Three problems that testing caught, and how I fixed them

Each was found by the evaluation or the demo, not by reading the code

1 · "Car insurance" scored 0.52

Vector similarity can't detect off-topic questions. **Fix:** an LLM grader with a strict "directly answers" rule, plus a retry loop and fallback.

2 · Structured output was unreliable

Ollama's JSON-schema mode returned null for "nurse" and the income, so I switched to tool calling. But the model sometimes skips the tool call (88% accuracy). **Fix:** retry, then JSON mode, then a safe default, which reached 100%.

3 · "Hi, I'm Ploy" started a lead

A self-introduction isn't interest. **Fix:** details count as a lead only after interest, or when a phone number is given. A Thai demo turn caught the first version of the fix.

The lesson: with LLM systems, an evaluation set and a replayable demo are what make changes safe.

NEXT STEPS

From working prototype to production

What I would do with the real documents and more time

- 1 Real PDFs:** re-ingest, OCR scanned Thai pages, and rebuild the evaluation set with the sales team.
- 2 Better retrieval:** hybrid BM25 + vector search and a cross-encoder reranker for long documents.
- 3 Monitoring:** LangSmith or OpenTelemetry tracing of every node, plus answer feedback buttons in the chat.
- 4 Security & PDPA:** log-in per sales agent, encrypt leads, consent text before collecting details, retention rules.
- 5 Deployment:** Docker Compose (Ollama + API + UI), or a hosted model if latency matters more than locality.
- 6 CRM:** push leads from the MCP tool straight into the CRM instead of SQLite.

THANK YOU

Questions? Let's try it live.

Run **python main.py serve** (or start_chatbot.bat) and open <http://127.0.0.1:8000>. Source, README, demo logs and evaluation are in Case2_GenAI_RAG.